

Ops: A Rules-Driven Life-Systems Scheduler & Retrospective Engine

Project Proposal for UCS503P

Thapar Institute of Engineering and Technology

Udaypratap Singh (1024030698), CSED

Daksh Jand (1024030936), CSED

Saksham (1024030332), CSED

Raunak Mahajan (1024030837), CSED

August 2026

1 Higher-Order Goal

...secondary framing

This project aims to eliminate cognitive load and planning friction in personal execution by treating time-management as a deterministic, constraint-satisfaction scheduling problem. Rather than relying on passive, unstructured to-do lists that demand constant willpower, the immediate engineering objective is to translate real-world human operating constraints into an automated, interactive, and measurable software dashboard.

2 Time-to-Value

...primary heuristic

We prioritize rapid increments and fast feedback over exhaustive speculative design:

- Deliver the standalone constraint-scheduling engine and interactive 50-minute grid canvas within the first milestone.
- Use weekly iterations backed by automated testing and CI pipelines to validate slotting algorithms and prevent regression.
- Measure practical daily utility immediately on real schedules rather than waiting for an arbitrary end-of-semester milestone.

3 Problem Statement

Individuals face structural failure modes when trying to maintain complex routines (e.g., coursework, skill development, fitness quotas, projects) using standard productivity tools:

- **Infinite Horizon Fallacy:** Checkbox lists ignore calendar geometry and treat human capacity as unconstrained, leading to systematic over-commitment and burnout.
- **High Manual Friction:** Manually rebuilding daily calendars when tasks slip consumes excessive willpower, causing users to abandon time-blocking entirely.
- **Fragile Streak Metrics:** Traditional habit trackers enforce binary daily streaks; missing a single day invalidates the entire chain, causing disproportionate demoralization.
- **Lack of Execution Feedback:** Standard calendars do not perform automatic post-hoc reconciliations comparing planned allocations against actual logged execution.

Why this matters now (contextual relevance): Balancing rigorous engineering curricula with independent technical growth and health demands an automated operational system that optimizes schedule placement dynamically while tolerating real-world disruptions.

4 Proposed Solution

4.1 Overview ...*Software Proposition*

Build **Ops**, a rules-driven, single-user life-operating engine consisting of:

- **50-Minute Block Canvas:** Interactive weekly timeline discretized into standard 50-minute focus blocks with built-in transition buffers.
- **Deterministic Constraint Scheduler:** Automatic engine that slots flexible tasks around fixed calendar constraints according to rule priority and rest buffers.
- **Rolling Consistency Engine:** Non-punitive tracking tracking weekly volume targets (e.g., 4 gym sessions/week) rather than unbroken daily chains.
- **Retrospective Generator:** Automated weekly summaries analyzing planned vs. executed blocks and scheduling drift.

4.2 Core Workflow

1. User defines fixed commitments (classes, labs) and recurring target quotas (gym, study blocks).
2. Solver generates a validated weekly schedule placing flexible tasks into open capacity.
3. User executes blocks and logs completion or performs drag-and-drop manual adjustments.
4. At week's end, the system computes compliance rates, maintains aggregate streaks, and highlights scheduling bottlenecks.

4.3 Operational Constraints for an Applied Academic Setting

- Scope bounded strictly to single-user execution; omit multi-tenant auth and external calendar synchronization to maximize core engine depth.
- Implement the constraint solver as a pure Python algorithm without heavy black-box solver dependencies to preserve algorithmic clarity and testability.

5 Solution Approach

...*Engineering focus*

The engineering focus centers on clean domain modeling, robust state reconciliation, and algorithmic slot allocation.

5.1 Constraint-Driven Slot Allocation

The scheduler operates via a deterministic, multi-pass heuristic:

- **Pass 1 (Immutable Constraints):** Map fixed academic events directly onto absolute grid coordinates.
- **Pass 2 (Quota Placement):** Allocate recurring targets respecting diurnal affinity (e.g., evening gym) and minimum spacing windows.
- **Pass 3 (Deadline Backtracking):** Schedule one-off deliverables backwards from hard due dates.
- **Pass 4 (Backlog Filling):** Pack remaining vacant blocks with ranked flexible tasks sorted by urgency-to-capacity ratios.

5.2 Web Architecture ...*web-based solution*

- **Frontend:** React with TypeScript and `dnd-kit` for accessible, collision-aware block manipulation on a 50-minute grid canvas.
- **Backend API:** FastAPI (Python 3.12) providing type-safe REST endpoints for schedule generation, commitment definitions, and execution logging.
- **Persistence:** PostgreSQL / SQLite managed via SQLAlchemy 2.0 and Alembic schema migrations.

5.3 CI/CD and Delivery Enablers

Automated GitHub Actions pipeline running unit test suites across solver constraints, ORM data contracts, and linting on every commit.

6 Evaluation Criteria

...measurable and attributable

6.1 Primary Evaluation Metric

Schedule Generation & Re-slot Latency (SGL): Time required to resolve and populate a 7-day schedule grid with ≥ 40 constraints.

- **Target:** Median SGL < 100 ms for real-time interactive canvas re-slotting.
- **Attribution:** Measured via backend profiling hooks during constraint re-computations.

6.2 Secondary Evaluation Metrics

- **Weekly Target Compliance Rate:** Proportion of defined weekly recurring quotas successfully completed ($\geq 85\%$ target).
- **Constraint Collision Rate:** Frequency of hard time overlaps produced by the solver (strictly 0 permissible).
- **Manual Override Frequency:** Number of manual drag-and-drop adjustments required post-generation to track solver accuracy.

6.3 Pilot Validation Plan ...*high-level*

Evaluate the platform across a multi-week active semester test phase, capturing execution logs and comparing scheduled allocations against actual completion rates.

7 Scalability & Maintainability

- **Architectural Decoupling:** Clear separation between the pure Python solver module and API transport layer enables isolated benchmarking.
- **Indexed Relational Queries:** Schema structured with compound indexes on timestamps and status flags to ensure sub-millisecond retrospective aggregations.
- **Containerized Deployment:** Standardized `Dockerfile` and `docker-compose` configuration for zero-setup replication.

8 Engine Availability Heuristic

The project leverages proven, standardized software components:

- FastAPI and Pydantic v2 for self-documenting, asynchronous REST endpoints.
- SQLAlchemy 2.0 ORM for relational data modeling and migration tracking.
- React 18 with `dnd-kit` for interactive drag-and-drop layout coordination.
- Pytest framework for property-based constraint verification.

9 Project Scope and Deliverables

9.1 Initial Deliverable ...*first iteration*

- Relational schema for commitments, constraint rules, and time blocks.
- Pure Python scheduling module capable of resolving fixed and recurring quotas.
- Basic REST endpoints for commitment ingestion and schedule retrieval.
- Automated CI pipeline enforcing backend unit tests.

9.2 Subsequent Deliverables ...*iteration-friendly*

- Interactive React 50-minute block timeline with drag-and-drop override handling.
- Rolling streak and aggregate consistency tracking module.
- Automated end-of-week review dashboard comparing planned vs. logged blocks.

10 Risks and Mitigations

- **Constraint Deadlocks:** Infeasible constraint combinations handled gracefully via priority fallback queues and unallocated block alerts.
- **UI Drag-and-Drop Complexity:** Utilize mature primitives (`dnd-kit`) with discrete 50-minute snap grids to avoid continuous-time coordinate bugs.
- **Scope Drift:** Maintain strict exclusion of multi-tenant auth and complex external calendar APIs during the 12-week development window.

11 Summary

This project proposes **Ops**, a rules-driven life-operating engine that automates personal time-blocking and consistency tracking. By grounding schedule generation in deterministic constraint solving and measuring rolling weekly quotas, the project demonstrates technical depth in algorithmic scheduling, state management, and full-stack software engineering.